

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · Sistemas Inteligentes

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Luana Beatriz Ribeiro Abrantes

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	4
4.1	Atividade 1 — Agenda telefônica em Flask	4
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	5
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	7
4.4	Atividade 4 — Controle de acesso por papel — RBAC	11
4.5	Atividade 5 — Logs e auditoria	14
4.6	Atividade 6 — Upload e perfil de usuário	16
5	Atividades extras (opcional)	18
5.1	Atividade E1 — Documentação Swagger/OpenAPI	18
5.2	Atividade E2 — CI/CD com GitHub Actions	20
5.3	Atividade E3 — Observabilidade — health checks e métricas	22
6	Considerações finais	25
7	Declaração de autoria	26

1 Introdução

A disciplina Introdução à Computação em Nuvem (ISW055) aborda o desenvolvimento e a publicação de aplicações, envolvendo contêineres, microsserviços, autenticação, controle de acesso e armazenamento de dados. Durante o bimestre, esses conceitos foram trabalhados em atividades práticas, permitindo acompanhar a evolução de uma aplicação e compreender a comunicação entre seus serviços.

A primeira atividade foi realizada em sala de aula e consistiu em uma agenda desenvolvida em Python com Flask, com cadastro e listagem de clientes e persistência em arquivo JSON. Em seguida, foi desenvolvido um catálogo de filmes de Tom Hanks com Node.js e Express, integrado à API TMDb e ao MariaDB para armazenar os dados dos usuários, favoritos e comentários. Nas etapas seguintes, o catálogo recebeu um microsserviço de autenticação, permissões para usuários e administradores, registros de auditoria e um perfil de usuário com upload de imagem para o MinIO.

Este relatório apresenta as seis atividades realizadas, descrevendo as funcionalidades implementadas, as dificuldades encontradas e as soluções adotadas. As entregas são acompanhadas de links e capturas de tela, enquanto a metodologia explica o processo de desenvolvimento e os critérios utilizados para registrar as datas de realização.

2 Metodologia

O desenvolvimento das atividades ocorreu de forma progressiva. A primeira prática foi realizada em sala de aula, com a construção de uma agenda em Python e Flask. A partir da segunda atividade, o trabalho se concentrou no projeto `catalogo-filmes`, desenvolvido no Visual Studio Code e versionado no GitHub. Cada etapa acrescentou funcionalidades ao catálogo, permitindo aplicar os conteúdos da disciplina e verificar sua integração com os recursos já implementados.

As funcionalidades foram verificadas por meio da execução local, de testes das rotas e da interação com as telas do sistema, incluindo cadastro, login, favoritos, comentários e permissões de acesso. O Docker Compose foi utilizado para configurar os contêineres e a comunicação entre os serviços, enquanto a publicação no servidor da disciplina foi gerenciada pelo Portainer. Os problemas encontrados durante a implementação foram investigados e corrigidos, com novas verificações após os ajustes.

As datas e os horários das atividades versionadas foram obtidos pelo histórico de commits do Git, considerando o fuso horário de Brasília (UTC-03). Os registros selecionados como referência são identificados nas fichas e acompanhados de links para consulta no GitHub. Para a atividade em Flask, realizada sem versionamento, foram utilizados a data e o horário informados da prática em sala de aula. A situação de cada entrega foi definida pela comparação entre a data registrada e o prazo previsto pelo professor.

A documentação de cada atividade reúne a descrição do trabalho realizado, as dificuldades encontradas, as soluções adotadas e as evidências correspondentes. As capturas de tela apresentam as interfaces, as configurações e os resultados observados, complementando os registros do repositório e permitindo verificar os recursos descritos no relatório.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Aplicação web em Python com Flask para cadastro e listagem de clientes, com persistência em arquivo JSON.	07/08/2026	07/08/2026 21:00	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e favoritos e comentários separados por usuário.	20/08/2026	20/08/2026 13:49	entregue
3	Desacoplando o login — microserviço de autenticação	Separação da autenticação do catálogo em um microserviço responsável por cadastro, login, validação de sessão e recuperação de senha.	28/08/2026	28/08/2026, às 18h20 (UTC-03)	Entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	04/09/2026 18:16:15 (UTC-03:00)	entregue
5	Logs e auditoria	Implementação de um microserviço de auditoria com Redis Streams para registrar eventos da aplicação e disponibilizar sua consulta somente a administradores.	25/09/2026	24/09/2026 00:40:41 (UTC-03:00)	entregue
6	Upload e perfil de usuário	Implementação do perfil do usuário com foto, biografia e favoritos, upload de imagens para o MinIO e validação de propriedade no backend.	02/10/2026	02/10/2026 21:47:33 (UTC-03:00)	entregue
E1	Documentação Swagger/OpenAPI	Documentação Swagger/OpenAPI de pelo menos dois serviços e chamada real executada pelo Try it out.	sem prazo	07/10/2026 21:15:01 (UTC-03:00)	entregue
E2	CI/CD com GitHub Actions	Pipeline de testes, validação OpenAPI, builds e publicação de imagens no GHCR, com atualização manual no Portainer.	sem prazo	07/10/2026 22:20:50 (UTC-03:00)	concluída
E3	Observabilidade — health checks e métricas	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.	sem prazo	07/10/2026 23:15:00 (UTC-03:00)	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Aplicação web em Python com Flask para cadastro e listagem de clientes, com persistência em arquivo JSON.
Data planejada	07/08/2026
Data realizada	07/08/2026 21:00
Situação	entregue
Evidência	Realizada em sala de aula, sem versionamento — captura de tela da aplicação em funcionamento.
Link	Não se aplica — atividade local, sem versionamento.

O que foi feito.

Foi desenvolvida uma agenda de clientes em Python com Flask, reunindo o formulário de cadastro, a listagem e o processamento dos dados em uma aplicação monolítica. A interface permite informar nome, telefone e valor, consultar os registros em uma tabela e visualizar a quantidade total de cadastros.

A persistência foi realizada em um arquivo JSON, utilizado para salvar e recuperar os registros, sem necessidade de um banco de dados relacional. Além da interface no navegador, foram disponibilizadas as rotas GET /cadastro e POST /cadastro, permitindo consultar e inserir dados por requisições em formato JSON.

A atividade foi realizada em sala de aula no dia 07/08/2026, às 21h00, e não foi versionada no GitHub. A captura de tela apresenta o formulário e a listagem de clientes, documentando o resultado da aplicação.

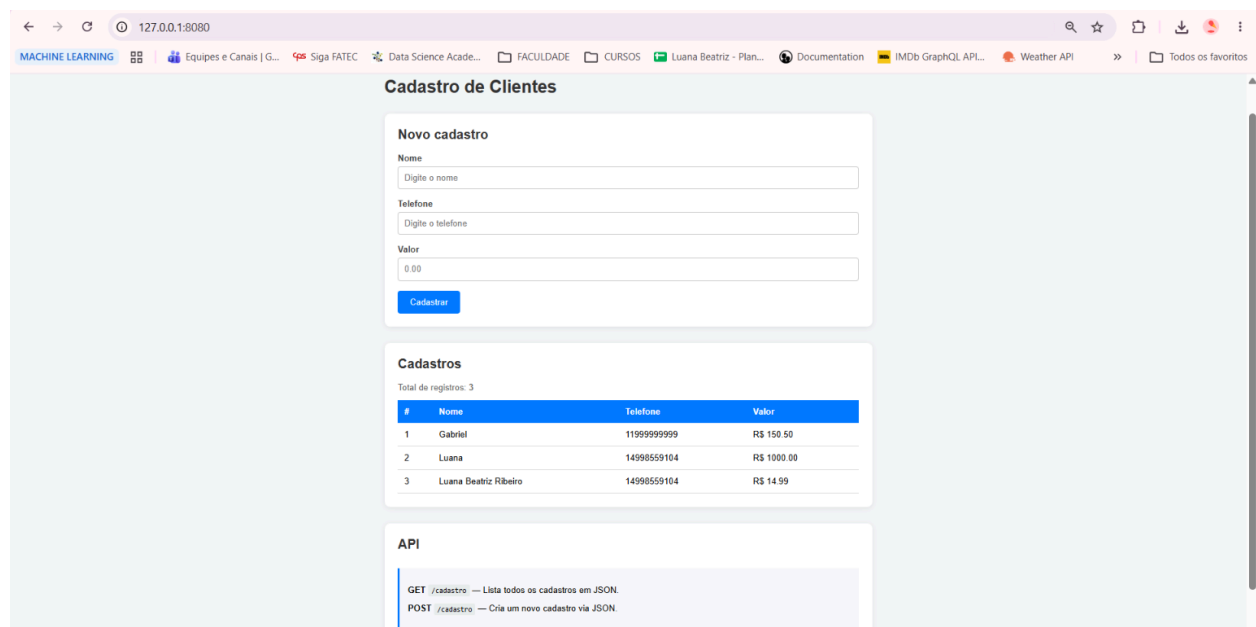


Figura 1: Atividade 1 — Aplicação Flask com formulário de cadastro e listagem de clientes.

Dificuldades e como foram resolvidas.

A principal dificuldade foi executar a aplicação Flask pelo terminal do Visual Studio Code. Houve dúvida sobre o comando necessário para iniciar o servidor e como acessar o sistema no navegador. A dificuldade

foi resolvida em sala de aula com o auxílio do professor, que explicou o procedimento de execução. Após a orientação, foi possível iniciar a aplicação e conferir o formulário e a listagem de clientes.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

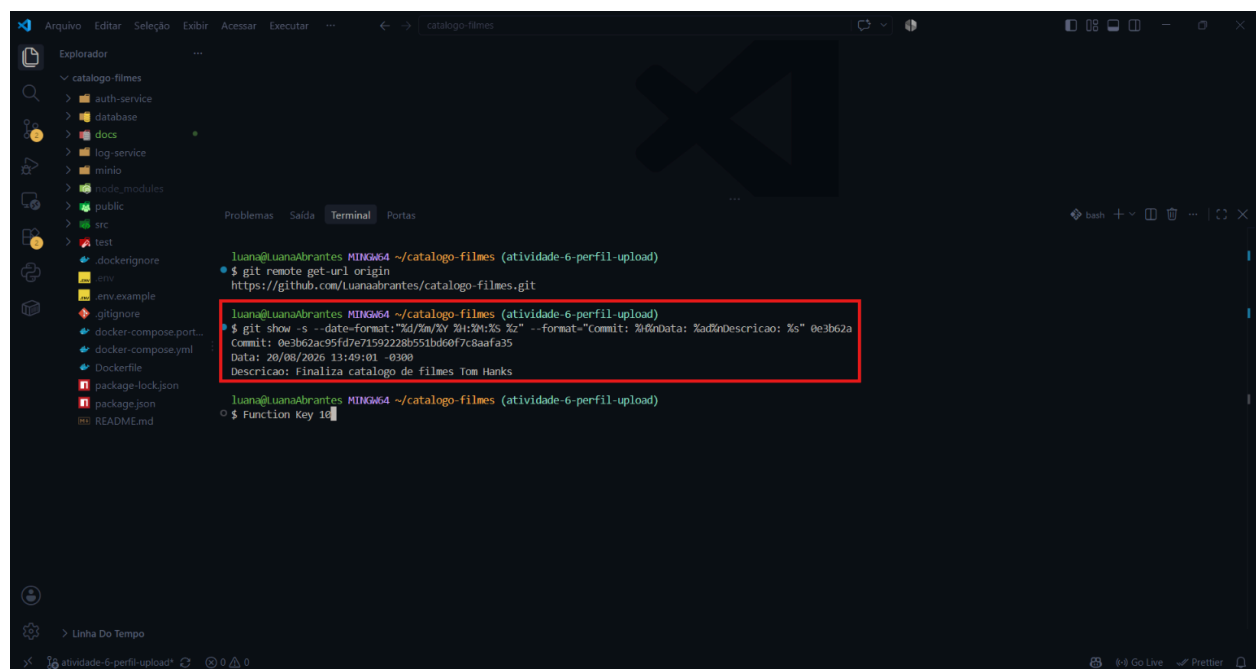
Descrição	Consumo da API TMDB, persistência em MariaDB e favoritos e comentários separados por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 13:49
Situação	entregue
Evidência	GitHub — commit 0e3b62a, README e captura do catálogo em funcionamento.
Link	https://github.com/Luanaabrantest/catalogo-filmes/commit/0e3b62a

O que foi feito.

Foi desenvolvida uma aplicação web para consultar filmes de Tom Hanks, utilizando Node.js e Express no backend e integração com a API TMDB para obter os dados apresentados no catálogo. A interface organiza os filmes em cartões com cartaz, título, ano de lançamento e sinopse, além de permitir que o usuário marque seus favoritos e publique comentários.

A persistência dos dados foi implementada no MariaDB, com armazenamento das contas de usuários, dos favoritos e dos comentários. Cada favorito foi vinculado ao usuário autenticado, mantendo uma seleção individual por conta, enquanto os comentários foram associados ao seu autor e ao filme correspondente.

Para disponibilizar a aplicação no servidor da disciplina, o projeto foi empacotado em uma imagem Docker e implantado pelo Portainer. Após a publicação, foram verificadas a exibição dos filmes e as operações de favoritar e comentar, confirmando a integração entre a interface, o backend e o banco de dados.



```

luana@luanaabrantest MINGW64 ~/catalogo-filmes (atividade-6-perfil-upload)
$ git remote get-url origin
https://github.com/Luanaabrantest/catalogo-filmes.git

luana@luanaabrantest MINGW64 ~/catalogo-filmes (atividade-6-perfil-upload)
$ git show -s --date-format:"%d/%m/%Y %H:%M:%S %z" --format="Commit: %h%nData: %ad%nDescrição: %s" 0e3b62a
Commit: 0e3b62ac95fd7e71592228b551bd60f7c8aafa35
Data: 20/08/2026 13:49:01 -0300
Descrição: Finaliza catalogo de filmes Tom Hanks

luana@luanaabrantest MINGW64 ~/catalogo-filmes (atividade-6-perfil-upload)
$ Function Key 16

```

Figura 2: Atividade 2 — Registro do commit da entrega, com identificação do repositório, data e horário.

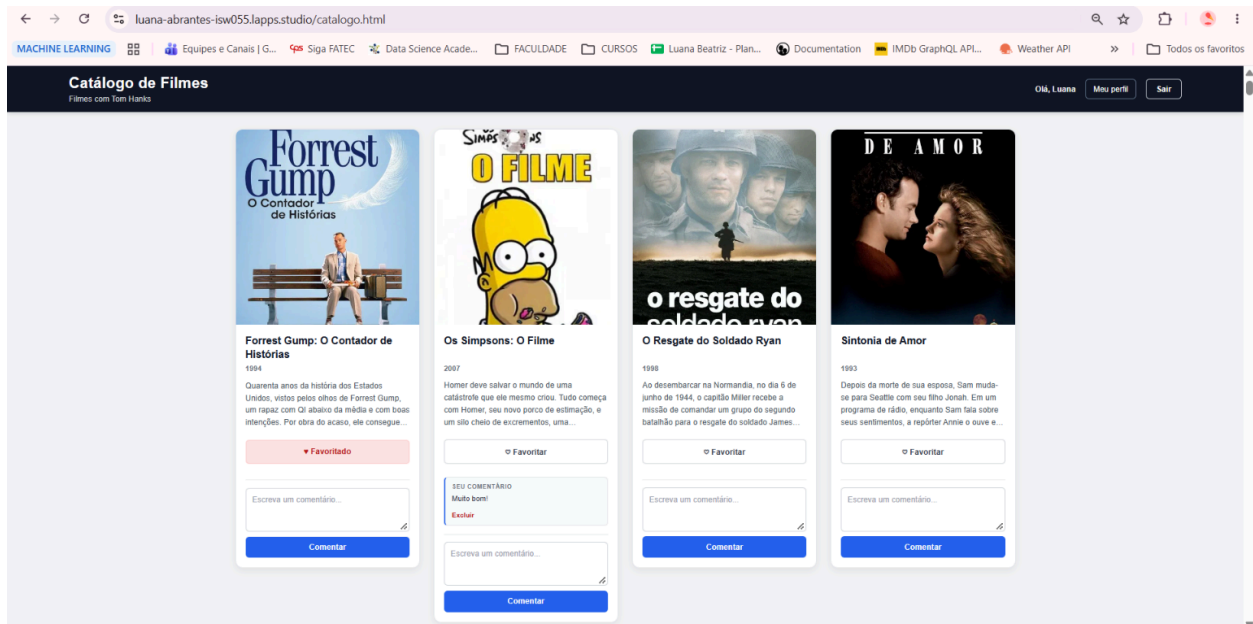


Figura 3: Atividade 2 — Catálogo em funcionamento, com filme favoritado e comentário publicado.

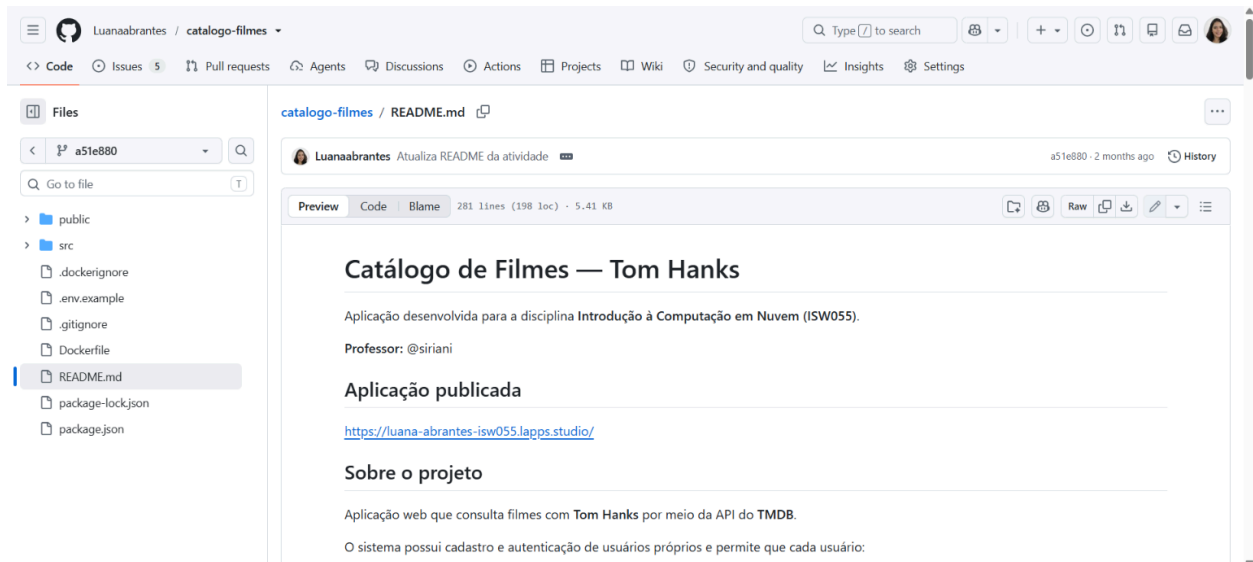


Figura 4: Atividade 2 — README do repositório com a apresentação do catálogo, referência à disciplina e endereço da aplicação publicada.

Dificuldades e como foram resolvidas.

A atividade envolveu autenticação, integração com a TMDB e deploy da aplicação. Uma das dificuldades foi configurar corretamente a autenticação dos usuários, utilizando bcryptjs para proteger as senhas, JWT para autenticação e cookies HttpOnly para manter a sessão. Também foi necessário entender o fluxo de consulta da API do TMDB, primeiro buscando o identificador de Tom Hanks e depois seus créditos em filmes, mantendo essas requisições no backend. No deploy, a principal dificuldade foi atualizar corretamente a aplicação no Portainer após alterações no projeto. Para isso, a imagem Docker foi recriada, enviada ao Docker Hub e o container foi atualizado com as variáveis de ambiente e portas configuradas corretamente.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Separação da autenticação do catálogo em um microsserviço responsável por cadastro, login, validação de sessão e recuperação de senha.
Data planejada	28/08/2026
Data realizada	28/08/2026, às 18h20 (UTC-03)
Situação	Entregue
Evidência	Histórico de commits, configuração Docker Compose e capturas das telas de login e cadastro.
Link	https://github.com/Luanaabrantest/catalogo-filmes/commit/02542ae

O que foi feito.

A autenticação foi separada do catálogo de filmes e implementada em um microsserviço chamado `auth-service`, desenvolvido com Node.js e Express. Esse serviço passou a concentrar o cadastro de usuários, o login, a validação de sessão e a recuperação de senha. O catálogo manteve a interface acessada pelo usuário e passou a encaminhar as solicitações de autenticação ao serviço, preservando o acesso às funcionalidades de favoritos e comentários.

No cadastro e no login, foram utilizados `bcryptjs` para gerar e verificar os hashes das senhas e `JWT` para identificar o usuário autenticado. O token foi armazenado em um cookie `HttpOnly`, impedindo seu acesso direto pelo JavaScript da interface. A consulta da sessão permitia verificar a autenticação do usuário antes do acesso aos recursos protegidos.

A recuperação de senha foi implementada com envio de e-mail pelo Mailtrap, contendo um link para redefinição e um token com validade de 30 minutos. Após sua utilização, o token não podia ser reutilizado. Foram realizados testes de cadastro, login, consulta da sessão e redefinição de senha, incluindo tentativas com tokens inválidos ou já utilizados.

A execução dos serviços foi organizada pelo Docker Compose, com o catálogo e o `auth-service` em contêineres separados. O catálogo utilizava a porta 3000 e se comunicava com a autenticação pela rede interna do Docker, no endereço `http://auth-service:3001`. A porta do `auth-service` não era publicada no host, mantendo o acesso ao microsserviço pela comunicação interna entre os contêineres.

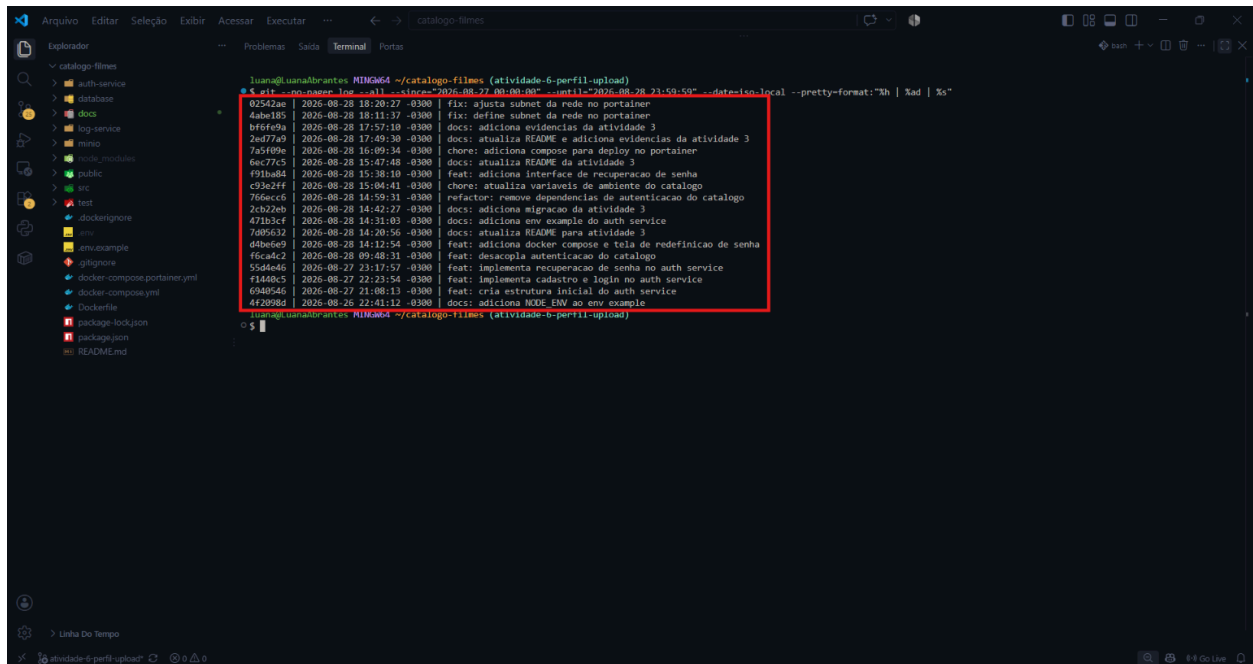


Figura 5: Atividade 3 — Histórico de commits referente à implementação do microserviço de autenticação.

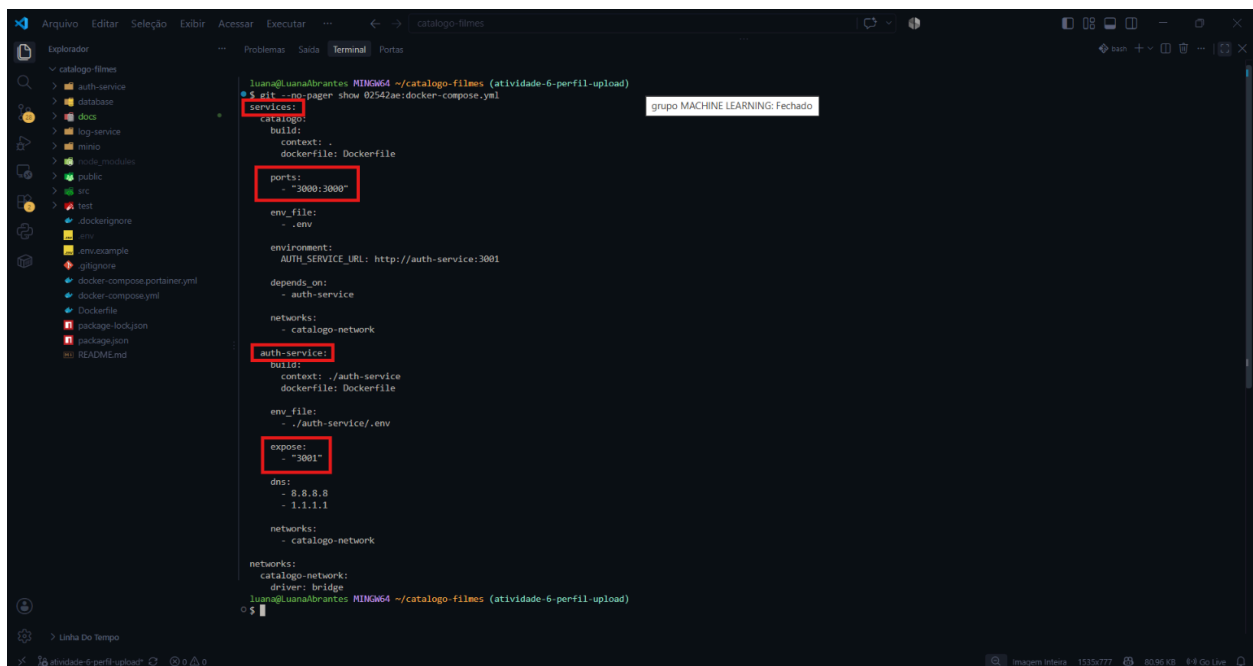
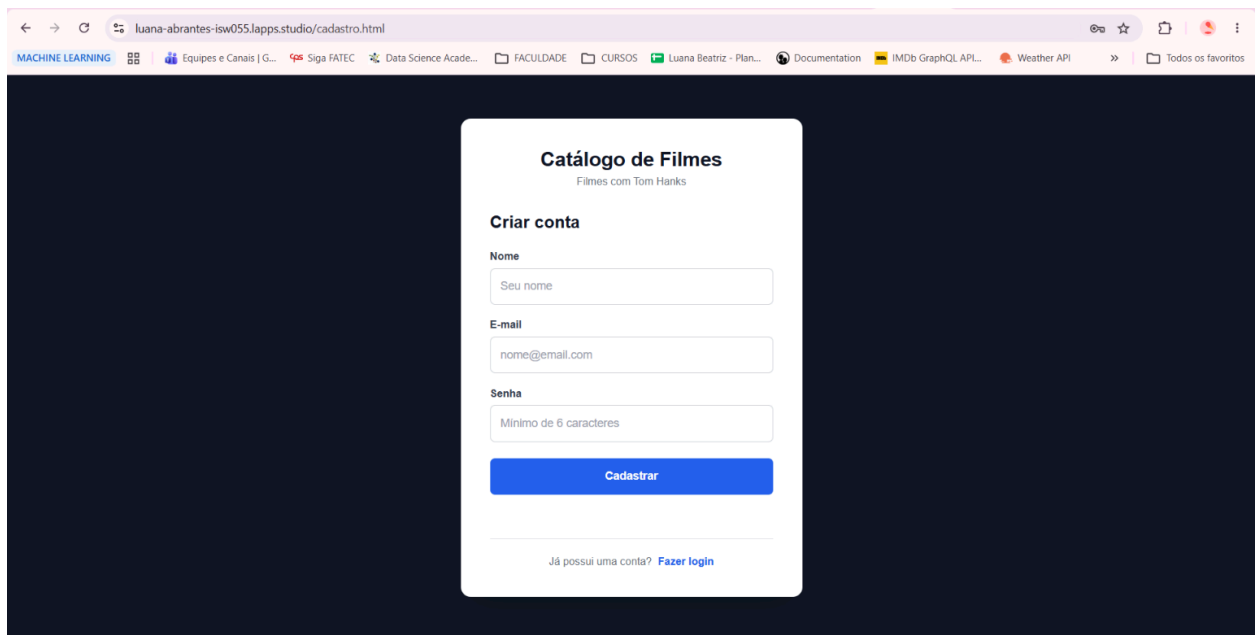
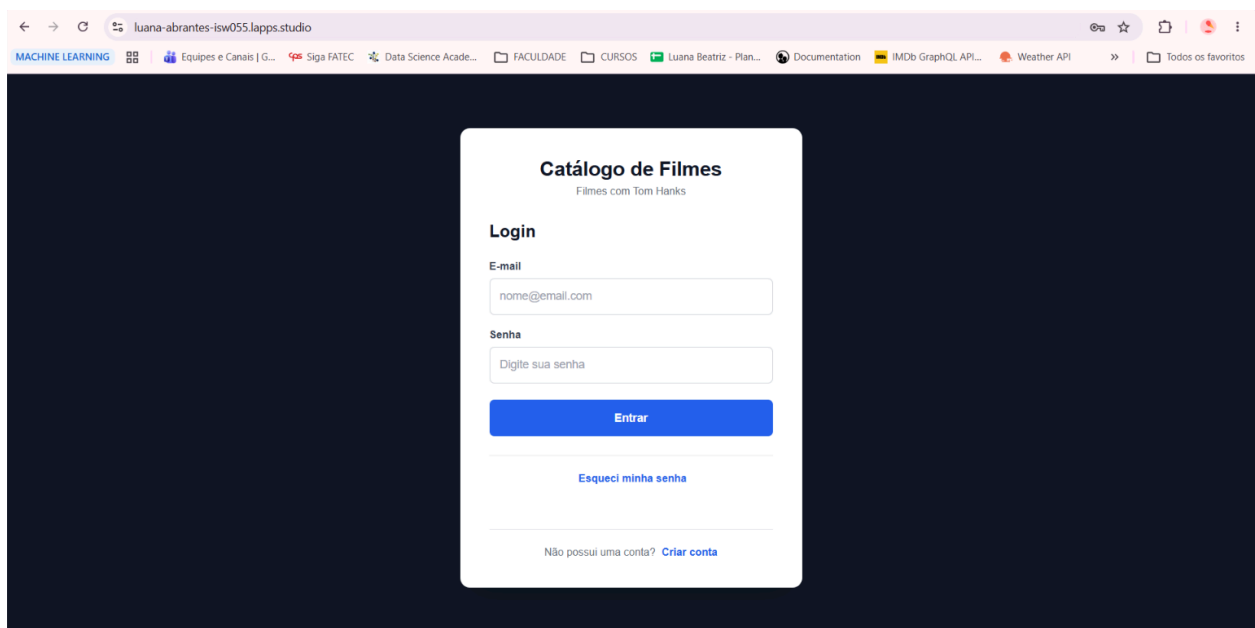


Figura 6: Atividade 3 — Configuração do Docker Compose com o catálogo e o serviço de autenticação em contêineres separados.



The screenshot shows a web browser window with the address bar displaying 'luana-abrantes-isw055.lapps.studio/cadastro.html'. The browser's tab bar includes 'MACHINE LEARNING', 'Equipes e Canais | G...', 'Siga FATEC', 'Data Science Acade...', 'FACULDADE', 'CURSOS', 'Luana Beatriz - Plan...', 'Documentation', 'IMDb GraphQL API...', 'Weather API', and 'Todos os favoritos'. The main content area features a dark blue background with a white registration form titled 'Catálogo de Filmes' and the subtitle 'Filmes com Tom Hanks'. The form is headed 'Criar conta' and contains three input fields: 'Nome' with the placeholder 'Seu nome', 'E-mail' with the placeholder 'nome@email.com', and 'Senha' with the placeholder 'Mínimo de 6 caracteres'. A blue 'Cadastrar' button is positioned below the fields. At the bottom of the form, a link reads 'Já possui uma conta? [Fazer login](#)'.

Figura 7: Atividade 3 — Interface de cadastro de usuários.



The screenshot shows the same web browser window as Figure 7, but the main content area displays a login form titled 'Catálogo de Filmes' with the subtitle 'Filmes com Tom Hanks'. The form is headed 'Login' and contains two input fields: 'E-mail' with the placeholder 'nome@email.com' and 'Senha' with the placeholder 'Digite sua senha'. A blue 'Entrar' button is located below the fields. Below the button is a link that reads 'Esqueci minha senha'. At the bottom of the form, a link reads 'Não possui uma conta? [Criar conta](#)'.

Figura 8: Atividade 3 — Interface de login com acesso ao cadastro e à recuperação de senha.

Catálogo de Filmes

Filmes com Tom Hanks

Recuperar senha

Informe o e-mail da sua conta. Você receberá um link para criar uma nova senha.

E-mail

Enviar link de recuperação

Se o e-mail estiver cadastrado, você receberá um link de recuperação.

[Voltar para o login](#)

Figura 9: Atividade 3 — Confirmação da solicitação de recuperação de senha.

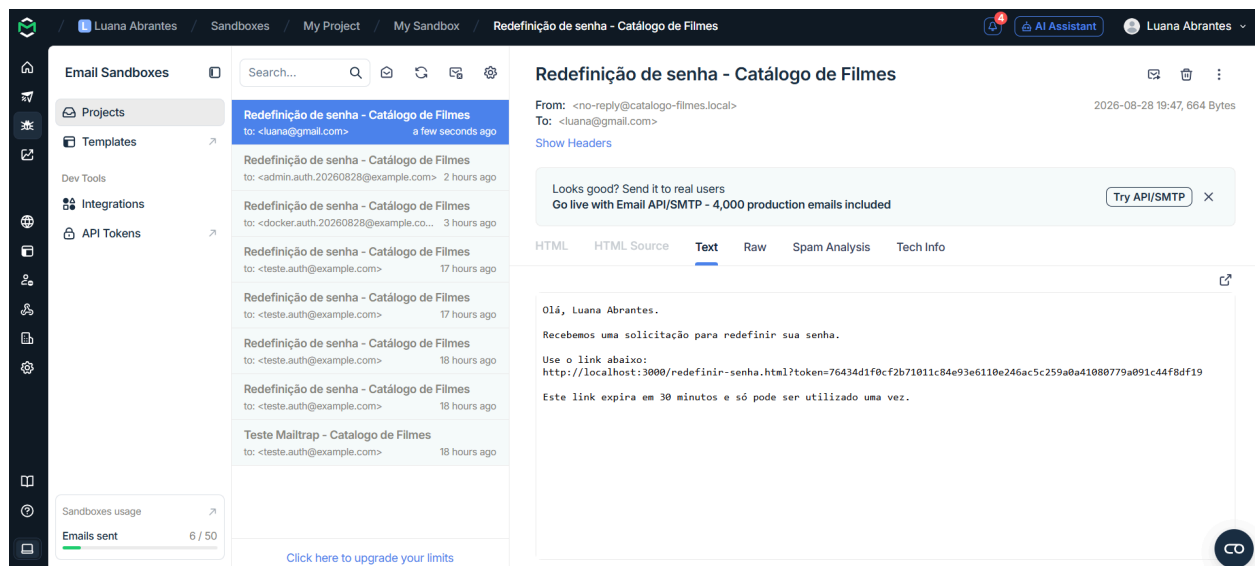


Figura 10: Atividade 3 — E-mail de recuperação recebido no Mailtrap, contendo o link para redefinição de senha.

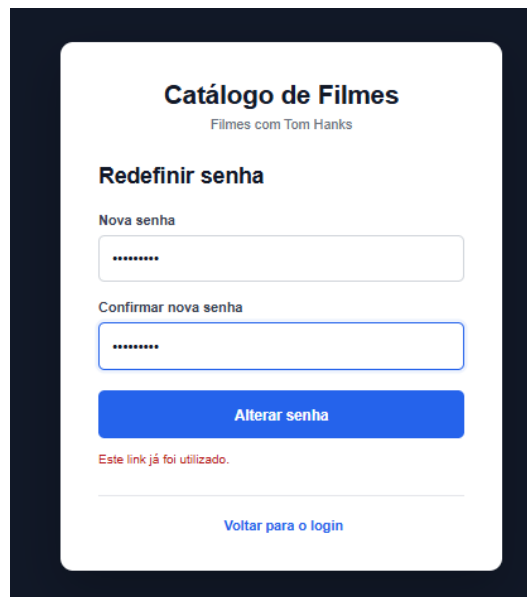


Figura 11: Atividade 3 — Rejeição de um link de recuperação de senha já utilizado.

Dificuldades e como foram resolvidas.

A principal dificuldade foi separar a autenticação do catálogo e transferi-la para um microserviço próprio, mantendo o catálogo como único ponto de entrada público. Também foi necessário configurar o fluxo de recuperação de senha, com geração de token aleatório, validade de 30 minutos, uso único e envio de e-mail. A integração do envio por SMTP exigiu ajustes nas variáveis de ambiente, primeiro utilizando o Mailtrap durante os testes e depois o Brevo no ambiente de produção, sem alterar a lógica principal da aplicação. Outra dificuldade ocorreu no deploy pelo Portainer, pois o Docker não conseguia criar automaticamente uma nova rede devido ao esgotamento e à sobreposição de sub-redes existentes no servidor. A solução foi definir manualmente uma sub-rede livre no `docker-compose.portainer.yml`, utilizando `10.88.0.0/24`.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	04/09/2026 18:16:15 (UTC-03:00)
Situação	entregue
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/Luanaabrantest/catalogo-filmes/commit/03f8d88ebcd2c413d841c5666d29d0aac0c661c5

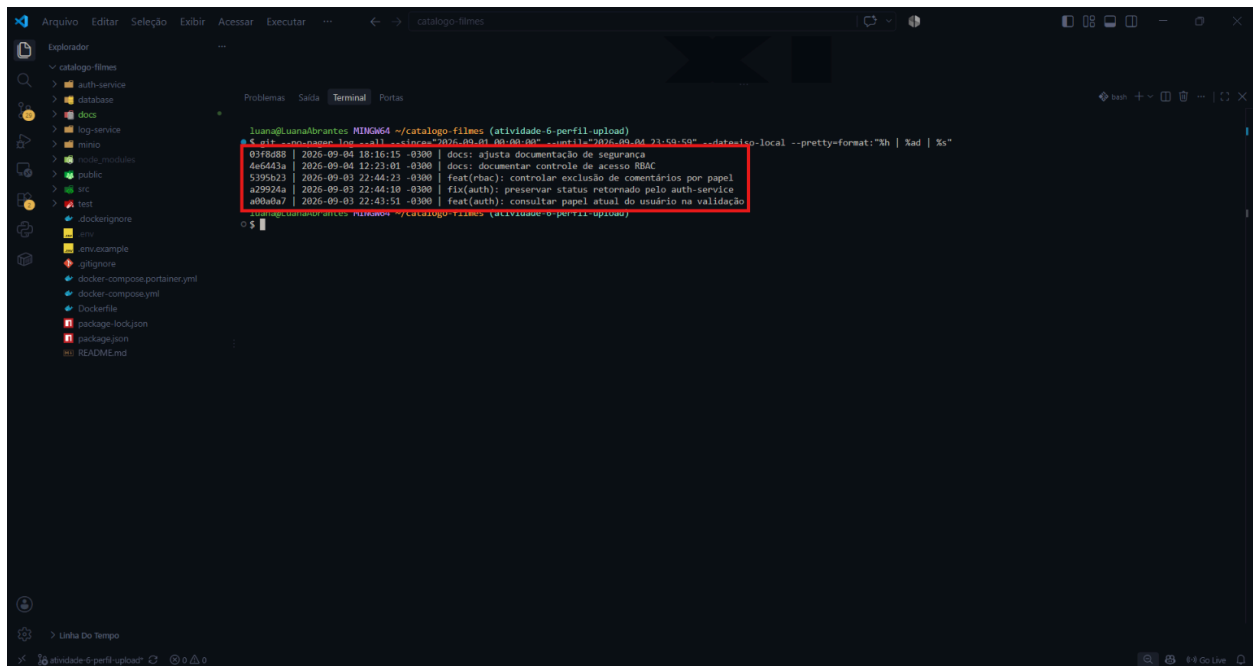
O que foi feito.

Foi implementado o controle de acesso por papéis (RBAC) no catálogo de filmes, com dois tipos de conta: `user`, para usuários comuns, e `admin`, para administradores. Ambos podem consultar o catálogo, favoritar filmes e publicar comentários. As operações administrativas passaram a exigir a validação do papel da conta no backend.

O auth-service foi ajustado para consultar o papel atual do usuário durante a validação da sessão e repassar essa informação ao catálogo. Dessa forma, a autorização considera a permissão registrada para a conta no momento da consulta, sem depender apenas do papel presente no token JWT.

Na exclusão de comentários, o backend verifica a identidade do usuário autenticado, seu papel e a autoria do comentário. A regra permite que o usuário comum remova somente os próprios comentários e que o administrador exclua comentários de qualquer usuário.

A validação das permissões foi feita no backend, de forma que tentativas de acesso sem autorização retornem 403 Forbidden, mesmo quando a requisição é realizada diretamente pela API. A demonstração prática utilizou duas contas para acessar a mesma rota, /api/admin/usuarios: a conta comum recebeu HTTP 403 Forbidden, enquanto a administradora recebeu HTTP 200 OK, com acesso à listagem de usuários.



```
luana@luanaabranes MINGW64 ~/catalogo-filmes (atividade-6-perfil-upload)
$ git --no-pager log --all --since="2026-09-01 00:00:00" --until="2026-09-04 23:59:59" --dateiso-local --pretty=format:"%h | %ad | %s"
03f8688 | 2026-09-04 18:16:15 -0300 | docs: ajusta documentação de segurança
4e6d43a | 2026-09-04 12:23:01 -0300 | docs: documentar controle de acesso RBAC
5395b23 | 2026-09-03 22:44:23 -0300 | feat(rbac): controlar exclusão de comentários por papel
a29924a | 2026-09-03 22:44:10 -0300 | fix(auth): preservar status retornado pelo auth-service
a08a9a7 | 2026-09-03 22:43:51 -0300 | feat(auth): consultar papel atual do usuário na validação
```

Figura 12: Atividade 4 — Histórico de commits da implementação do controle de acesso por papel.

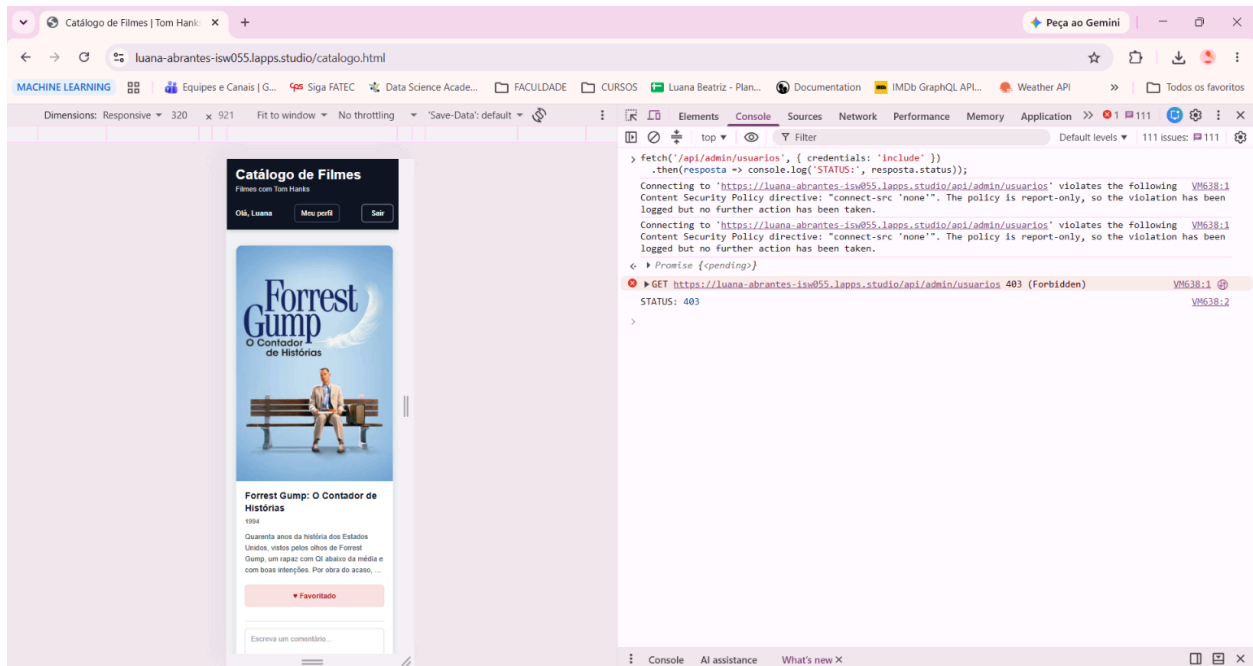


Figura 13: Atividade 4 — acesso à rota `/api/admin/usuarios` negado para usuário comum, com retorno HTTP 403 Forbidden.

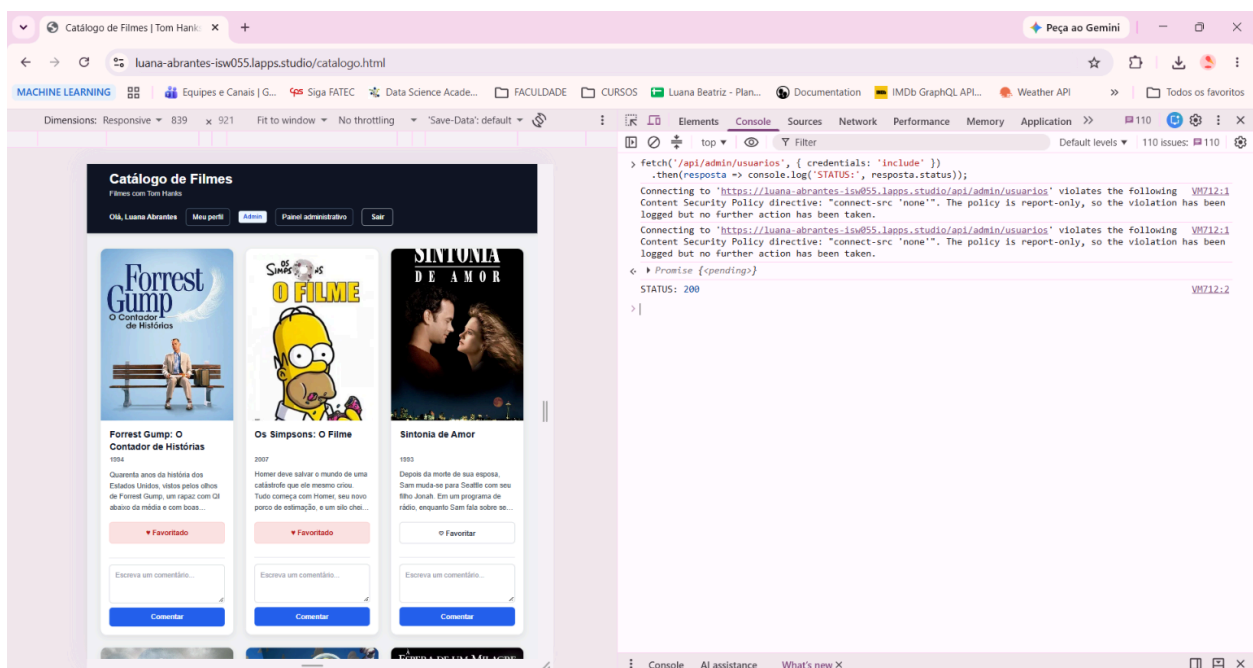


Figura 14: Atividade 4 — acesso à mesma rota `/api/admin/usuarios` permitido para administrador, com retorno HTTP 200 OK.

Dificuldades e como foram resolvidas.

A principal dificuldade foi garantir que o controle de acesso não dependesse apenas da interface ou das informações presentes no token. Para resolver isso, o `auth-service` foi ajustado para consultar no banco de dados o papel atual do usuário durante a validação da sessão, permitindo que o backend utilizasse essa informação para decidir se a operação era permitida.

Também foi necessário corrigir o tratamento dos códigos de resposta entre o catálogo e o `auth-service`, pois alguns erros poderiam ser interpretados de forma incorreta. O middleware foi ajustado para preservar os códigos HTTP retornados pelo serviço de autenticação, permitindo diferenciar corretamente situações como usuário não autenticado e usuário autenticado sem permissão.

Para comprovar a restrição de acesso, a mesma rota administrativa foi consultada com uma conta comum e uma conta administradora. A comparação dos resultados confirmou a recusa da requisição para o usuário comum, com retorno 403 Forbidden, e a autorização para o administrador, com retorno 200 OK.

4.5 Atividade 5 — Logs e auditoria

Descrição	Implementação de um microserviço de auditoria com Redis Streams para registrar eventos da aplicação e disponibilizar sua consulta somente a administradores.
Data planejada	25/09/2026
Data realizada	24/09/2026 00:40:41 (UTC-03:00)
Situação	entregue
Evidência	GitHub — commit e capturas do painel administrativo e da consulta dos logs.
Link	https://github.com/Luanaabrantest/catalogo-filmes/commit/25a2304

O que foi feito.

Foi desenvolvido um novo microserviço, o log-service, responsável por registrar eventos de auditoria da aplicação utilizando Redis Streams. O serviço registra informações como usuário responsável, ação realizada, data e hora, contexto da operação e IP quando disponível. Foram auditadas ações como LOGIN, LOGOUT, favoritar e desfavoritar filmes, criação e exclusão de comentários e tentativas de acesso sem permissão, registradas como ACAO_NEGADA.

O log-service e o Redis permanecem acessíveis apenas pela rede interna do Docker, sem portas publicadas diretamente para o usuário. A consulta dos eventos foi protegida por RBAC, permitindo acesso somente a usuários com papel admin. Para facilitar essa consulta, foi criada uma área de Auditoria no painel administrativo, onde é possível visualizar, pesquisar e filtrar os eventos registrados. Também foi validado que usuários comuns recebem 403 Forbidden ao tentar acessar os logs administrativos, e essa tentativa é registrada automaticamente na auditoria.

```

luana@luanaabrantest:~/catalogo-filmes (atividade-6-perfil-upload)
$ git log --pretty=format:"%h | %s" | %s" 2f821fd...25a2304
2f821fd | 21/09/2026 20:09:20 -0300 | feat: cria estrutura inicial do log-service
03f9d75 | 21/09/2026 20:11:38 -0300 | feat: adiciona Redis ao serviço de auditoria
31a667c | 21/09/2026 20:23:38 -0300 | feat: adiciona gravação de eventos no Redis Stream
3a67b2a | 21/09/2026 20:29:02 -0300 | feat: registra login e logout na auditoria
407e97a | 21/09/2026 20:33:23 -0300 | feat: registra ações do catálogo na auditoria
cb0a276 | 21/09/2026 20:43:15 -0300 | feat: registra tentativas de acesso negadas
6a7289f | 22/09/2026 00:37:00 -0300 | feat: adiciona consulta interna dos logs de auditoria
e0998fe | 22/09/2026 00:45:16 -0300 | feat: adiciona consulta administrativa dos logs
b351fdb | 22/09/2026 00:55:31 -0300 | docs: documenta logs e auditoria da atividade 5
2214f50 | 22/09/2026 08:20:23 -0300 | chore: adiciona compose da atividade 5 para Portainer
7517fae | 22/09/2026 21:37:54 -0300 | feat: adiciona gestão administrativa de usuários
e0ef5f6 | 22/09/2026 21:52:50 -0300 | feat: adiciona proxy administrativo de usuários
4803359 | 23/09/2026 17:22:54 -0300 | feat: cria estrutura do painel administrativo
6847f71 | 24/09/2026 00:17:21 -0300 | feat: cria estrutura do painel administrativo
d906d2a | 24/09/2026 00:28:55 -0300 | feat: adiciona gestão de usuários ao painel
25a2304 | 24/09/2026 00:40:41 -0300 | feat: adiciona auditoria ao painel administrativo
luana@luanaabrantest:~/catalogo-filmes (atividade-6-perfil-upload)
$

```

Figura 15: Atividade 5 — histórico de commits da implementação do serviço de logs e auditoria, com hashes, datas e horários.

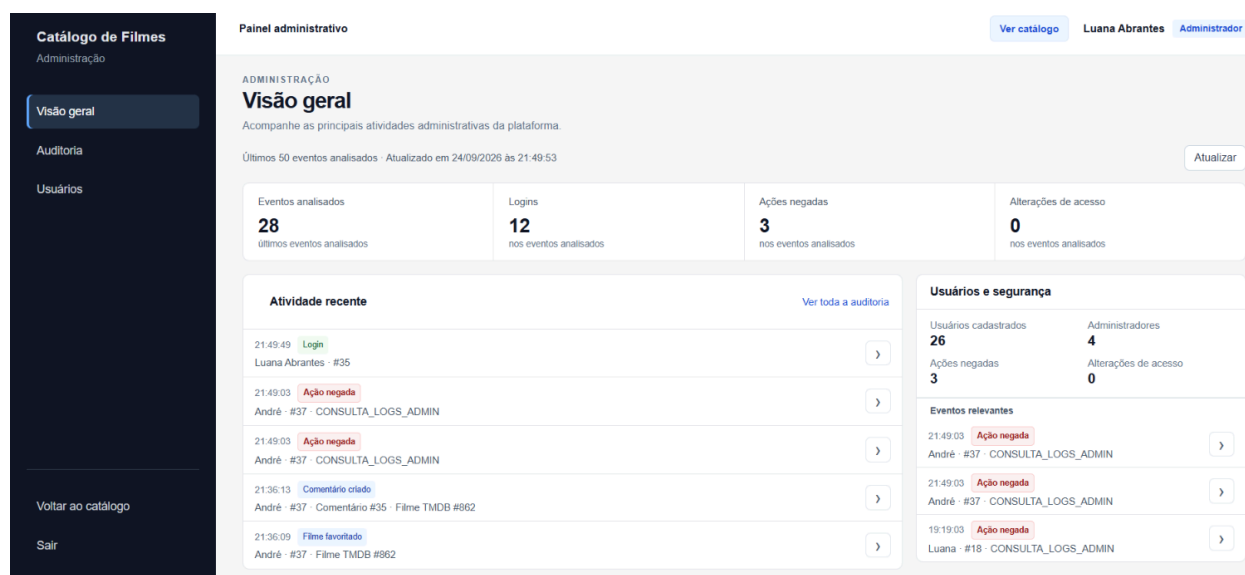


Figura 16: Atividade 5 — painel administrativo com resumo dos eventos de auditoria e registros de tentativas de acesso negadas à consulta de logs.

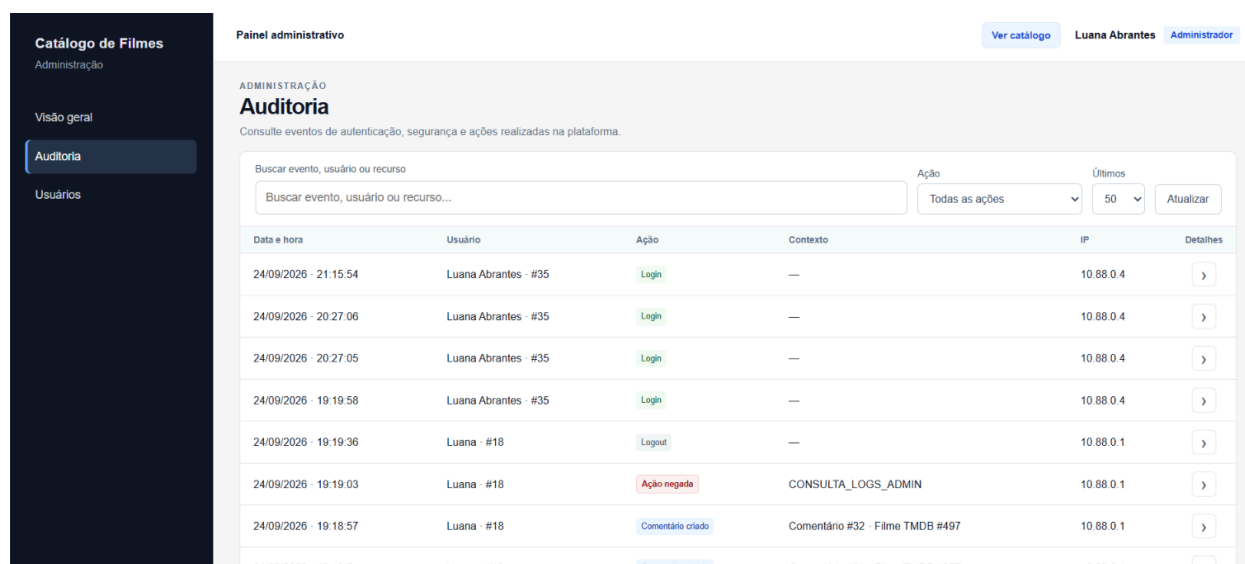


Figura 17: Atividade 5 — consulta dos registros de auditoria por uma conta administradora, com filtros e eventos de login, logout e tentativas de acesso negadas.

Dificuldades e como foram resolvidas.

Uma das principais dificuldades foi integrar o novo log-service ao catálogo e ao auth-service sem fazer com que uma falha no serviço de auditoria impedisse o funcionamento normal da aplicação. Para resolver isso, as chamadas de registro dos eventos foram implementadas com timeout e tratamento de erros, permitindo que ações como login, favoritos e comentários continuem funcionando mesmo se o serviço de logs estiver indisponível.

Outro ponto foi garantir que as tentativas de acesso sem permissão fossem registradas corretamente. A validação foi mantida no backend por meio do RBAC e as respostas 403 Forbidden passaram a gerar eventos `ACA0_NEGADA`, evitando depender apenas da interface.

Também foi necessário configurar a persistência dos eventos no Redis, utilizando Redis Streams, AOF e um volume Docker, para que os registros não fossem perdidos ao reiniciar os containers.

Por fim, a consulta dos logs foi protegida para permitir acesso somente a administradores, mantendo o log-service e o Redis acessíveis apenas pela rede interna do Docker.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Implementação do perfil do usuário com foto, biografia e favoritos, upload de imagens para o MinIO e validação de propriedade no backend.
Data planejada	02/10/2026
Data realizada	02/10/2026 21:47:33 (UTC-03:00)
Situação	entregue
Evidência	GitHub — commit, print do perfil com foto e print da tentativa de edição de outro perfil recusada.
Link	https://github.com/Luanaabrantess/catalogo-filmes/commit/b47bef1

O que foi feito.

Foi implementada uma página de perfil para cada usuário, contendo nome, foto, bio e filmes favoritados. O upload da foto foi integrado ao MinIO, utilizado como object storage, enquanto o MariaDB armazena somente a chave de referência do objeto na tabela perfis, evitando o armazenamento do arquivo binário diretamente no banco.

O upload aceita imagens nos formatos JPG, PNG e WebP, com limite de 5 MB. A aplicação valida o arquivo antes do envio e gera uma chave própria para armazenamento no MinIO. Para exibição da imagem foi adotada a estratégia de leitura pública do bucket, mantendo a escrita restrita ao backend.

Também foi implementado o controle de propriedade do perfil. O backend utiliza a identidade obtida pela sessão autenticada e impede que um usuário altere o perfil de outra pessoa. Tentativas desse tipo retornam 403 Forbidden e são registradas na auditoria como ACAO_NEGADA.

A página de perfil reaproveita os favoritos já existentes no sistema e apresenta os filmes em formato de cards. A atividade também adicionou o MinIO ao Docker Compose, incluiu persistência por volume e preservou os serviços já existentes de autenticação, RBAC, auditoria e Redis. Ao final, foram executados 49 testes automatizados, todos aprovados.

```

luana@LuanaAbrantes MINGW64 ~/catalogo-filmes (atividade-6-perfil-upload)
$ git --no-pager log --reverse --date=format: "%d/%m/%Y %H:%M:%S %z" --pretty=format: "%h | %ad | %s" 66a9996^..b47bef1
66a9996 | 30/09/2026 22:46:42 -0300 | feat: adiciona estrutura de perfil de usuario
b291a5d | 01/10/2026 08:29:35 -0300 | feat: adiciona MinIO ao ambiente Docker
5c8154c | 01/10/2026 08:32:50 -0300 | feat: integra catalogo ao MinIO
9890e0d | 01/10/2026 08:34:52 -0300 | feat: implementa upload de foto de perfil
11f1746 | 01/10/2026 08:38:43 -0300 | feat: protege edicao de perfil por proprietario e audita alteracoes
4d12a6b | 01/10/2026 17:05:17 -0300 | test: adiciona testes de perfil e upload
ad54256 | 01/10/2026 17:12:47 -0300 | build: compila MinIO oficial e valida integracao de perfil
7ff8345 | 02/10/2026 21:41:40 -0300 | feat: cria pagina de perfil com favoritos
b47bef1 | 02/10/2026 21:47:33 -0300 | docs: documenta atividade de perfil e MinIO
luana@LuanaAbrantes MINGW64 ~/catalogo-filmes (atividade-6-perfil-upload)
$

```

Figura 18: Atividade 6 — histórico de commits da implementação do perfil e do upload de imagens com MinIO, com hashes, datas e horários.

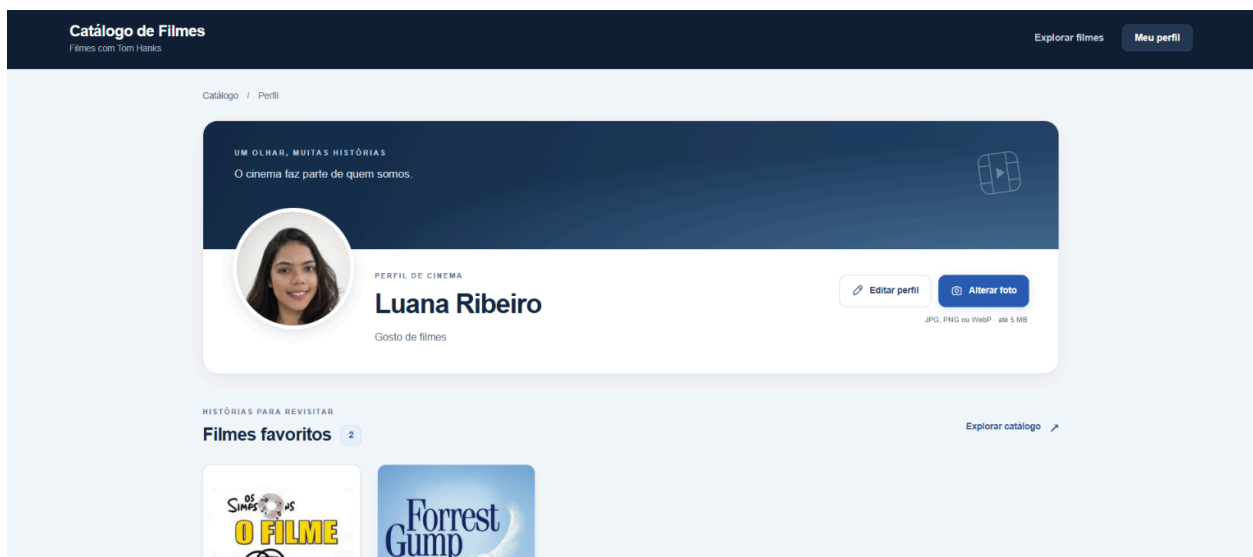


Figura 19: Atividade 6 — página de perfil do usuário com foto, biografia e filmes favoritos.

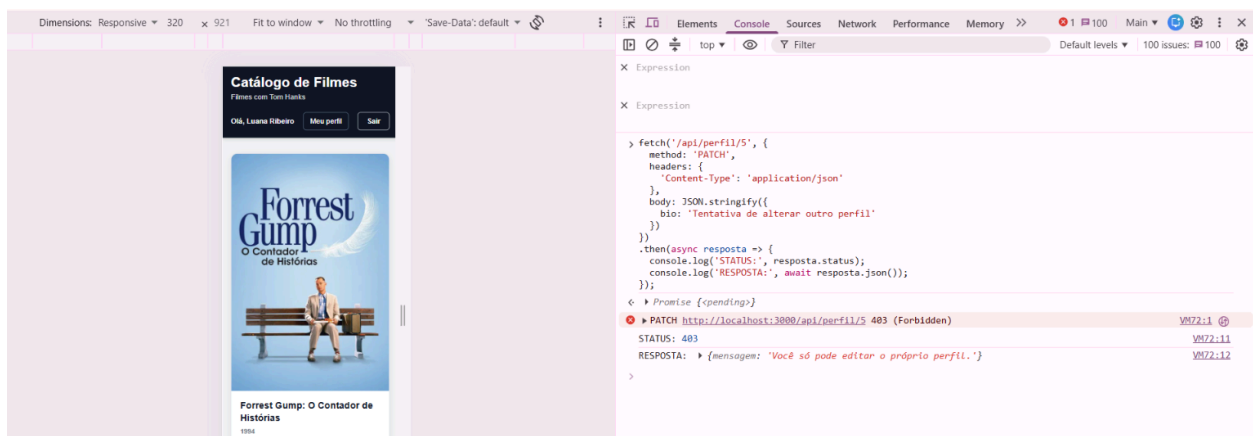


Figura 20: Atividade 6 — tentativa de alterar o perfil de outro usuário recusada pelo backend, com retorno HTTP 403 Forbidden.

Dificuldades e como foram resolvidas.

A principal dificuldade foi integrar o upload da foto de perfil com o MinIO sem armazenar a imagem diretamente no banco de dados. Foi necessário configurar o MinIO no docker-compose, criar as variáveis de ambiente e garantir que o catálogo conseguisse acessar o serviço pela rede interna do Docker. Durante os testes, a aplicação inicialmente não subia corretamente porque a variável `MINIO_SECRET_KEY` não estava definida no arquivo `.env`. O problema foi identificado pelo erro do Docker Compose e resolvido com a configuração das variáveis necessárias para o MinIO.

Outra dificuldade ocorreu quando a página de perfil retornava erro 503. A investigação mostrou que a tabela `perfis` ainda não existia no MariaDB, pois a migração da Atividade 6 ainda não havia sido aplicada. Após executar a migração e validar a estrutura da tabela, a rota `/api/perfil/me` passou a retornar 200 normalmente.

Também foi necessário garantir que um usuário não pudesse editar o perfil de outra pessoa. A solução foi realizar essa validação no backend, comparando o ID presente na rota com o ID do usuário autenticado obtido pela sessão. Quando os IDs são diferentes, a aplicação retorna 403 Forbidden e registra a tentativa como `ACAO_NEGADA` na auditoria.

5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Documentação Swagger/OpenAPI de pelo menos dois serviços e chamada real executada pelo Try it out.
Data planejada	sem prazo
Data realizada	07/10/2026 21:15:01 (UTC-03:00)
Situação	entregue
Evidência	Repositório público, commit com data e horário, README com referência ao professor e prints do Swagger.
Link	https://github.com/Luanaabrantest/catalogo-filmes/commit/780dd4f37f3c13471a8e285176372c805753b593

O que foi feito.

Foi implementada a documentação das APIs do catálogo e do auth-service, desenvolvidos em Node.js com Express, utilizando `swagger-jsdoc` e `swagger-ui-express`. Os contratos OpenAPI 3.0.3 documentam 22 operações do catálogo e oito do serviço de autenticação, incluindo parâmetros, corpos de requisição, autenticação e respostas de sucesso e erro. As especificações foram disponibilizadas em `docs/openapi/catalogo.json` e `docs/openapi/auth.json`, com schemas reutilizáveis e anotações junto às rotas.

O catálogo documenta autenticação, filmes, favoritos, comentários, auditoria, administração de usuários e perfil com upload de foto. O auth-service documenta cadastro, login, validação de sessão, recuperação de senha, administração de usuários e verificação de funcionamento. O Swagger está disponível em `/apidocs/` e `/apidocs/auth/`, e as especificações são servidas em `/openapi/catalogo.json` e `/openapi/auth.json`. O serviço de autenticação permanece na rede interna Docker; o Try it out está desativado na página de suas rotas internas.

No ambiente publicado, foi executada a chamada GET /api/filmes pelo Try it out do Swagger do catálogo, com retorno HTTP 200 e os dados dos filmes. As capturas documentam a execução real na API pública do catálogo, os endpoints do serviço de autenticação e o commit da implementação. O README inclui os links de acesso, as especificações e a referência ao professor.

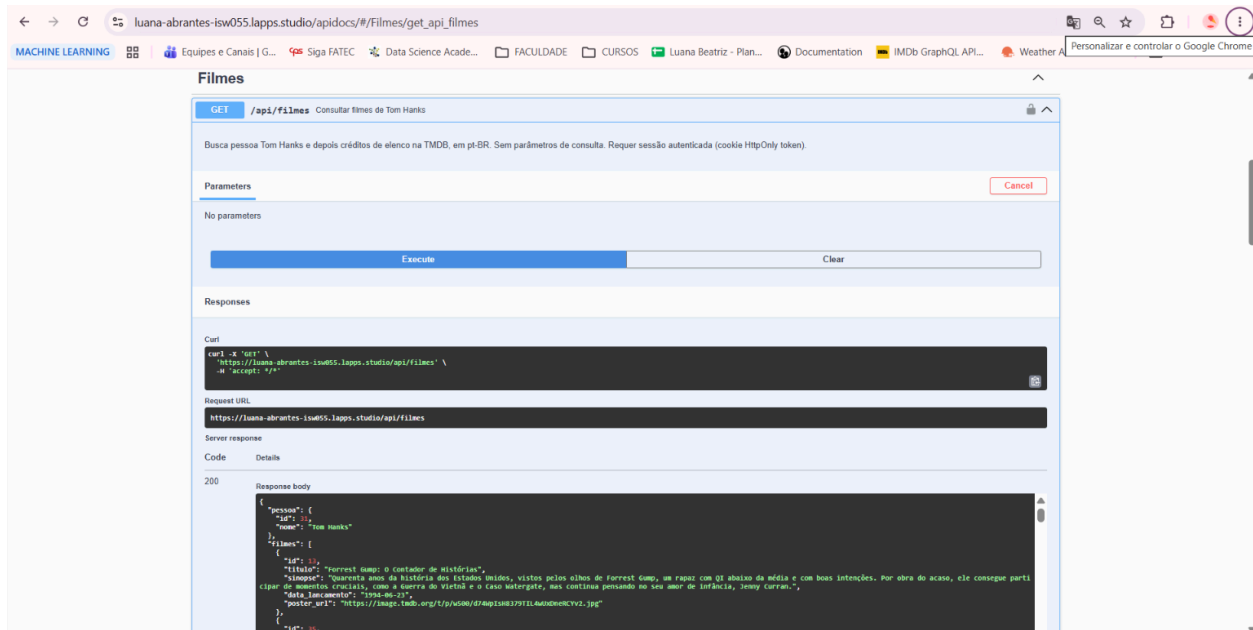


Figura 21: Atividade E1 — Execução real do GET /api/filmes pelo Swagger no ambiente publicado, com retorno HTTP 200 e dados dos filmes.

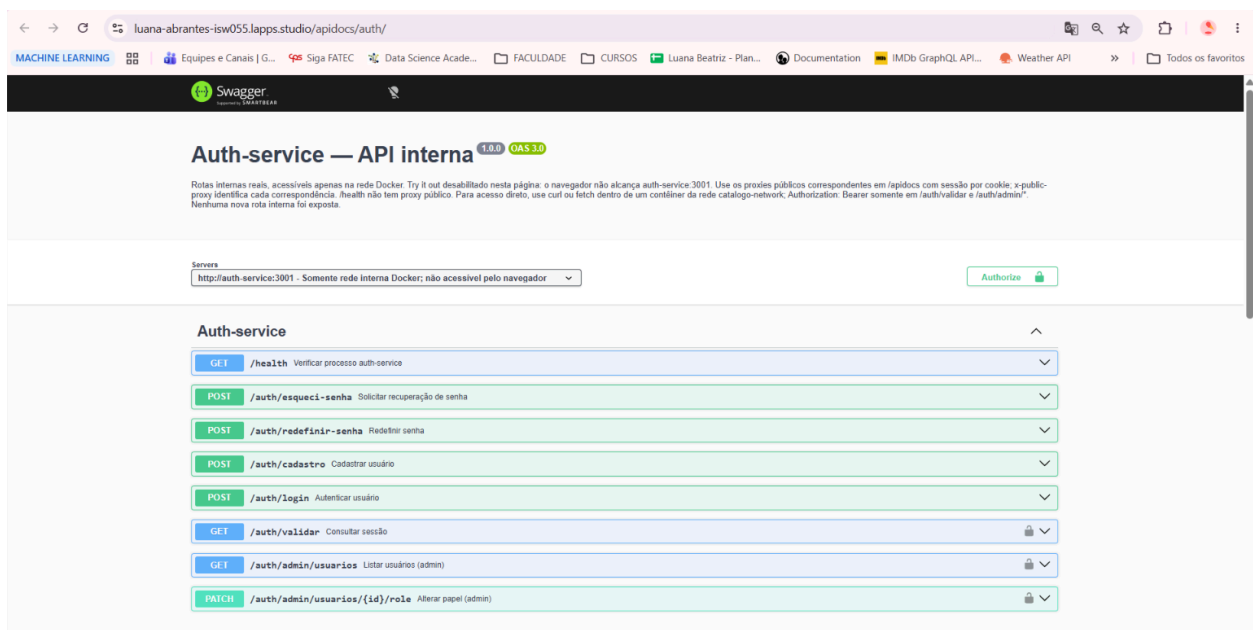


Figura 22: Atividade E1 — Documentação dos endpoints internos do auth-service no Swagger, com Try it out desativado.

```

luana@LuanaAbrantes MINGW64 ~/catalogo-filmes (atividade-extra-swagger)
$ git log -1 --format="Commit: %h%nData: %ad%nMensagem: %s" --date=iso-local
Commit: 780dd4f37f3c13471a8e285176372c805753b593
Data: 2026-10-07 21:15:01 -0300
Mensagem: docs: adiciona Swagger OpenAPI para catalogo e autenticao
luana@LuanaAbrantes MINGW64 ~/catalogo-filmes (atividade-extra-swagger)
$

```

Figura 23: Atividade E1 — Commit da implementação Swagger/OpenAPI, com hash, data e horário.

Dificuldades e como foram resolvidas.

Não houve dificuldades significativas durante a atividade. O trabalho exigiu atenção na organização da documentação e na conferência dos parâmetros, da autenticação e das respostas de sucesso e erro, para que as informações apresentadas no Swagger correspondessem ao funcionamento das APIs.

5.2 Atividade E2 — CI/CD com GitHub Actions

Descrição	Pipeline de testes, validação OpenAPI, builds e publicação de imagens no GHCR, com atualização manual no Portainer.
Data planejada	sem prazo
Data realizada	07/10/2026 22:20:50 (UTC-03:00)
Situação	concluída
Evidência	Branch atividade-extra-cicd; commit 0c6577452ec11940cc59339874b2dee2bb40c986; execução aprovada do GitHub Actions e capturas dos commits e containers no Portainer.
Link	https://github.com/Luanaabrantest/catalogo-filmes/commit/0c6577452ec11940cc59339874b2dee2bb40c986

O que foi feito.

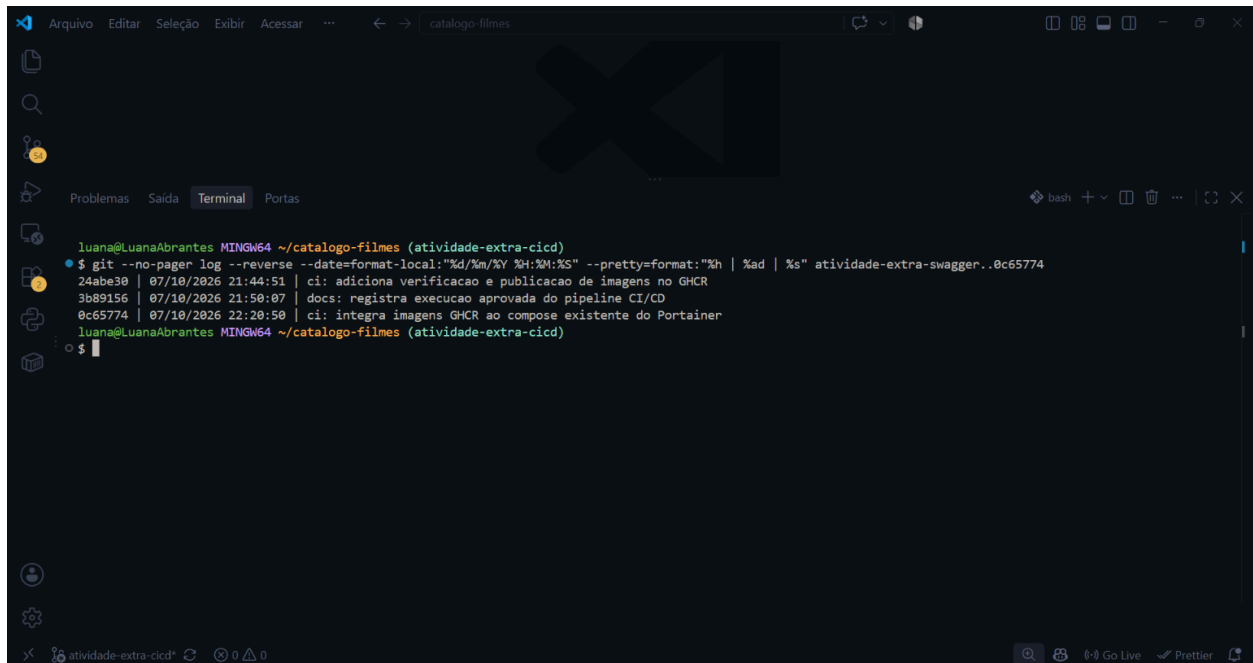
Foi criado o workflow `.github/workflows/cicd.yml`, executado a cada push e pull request. O pipeline instala as dependências do catálogo, do auth-service e do log-service com `npm ci`, executa os testes automatizados e valida a documentação OpenAPI. Em seguida, constrói e verifica as quatro imagens Docker: catálogo, autenticação, auditoria e MinIO. A etapa de publicação depende da aprovação das verificações, impedindo o envio das imagens quando algum teste ou build falha.

Nos eventos de push para a branch `atividade-extra-cicd`, as imagens verificadas são publicadas no GHCR com as tags `latest` e `sha-
<hash completo do commit>`, além do label `org.opencontainers.image.revision`. A publicação utiliza o `GITHUB_TOKEN`, com permissão `packages: write`. Os arquivos `.dockerignore` foram revisados para excluir credenciais das imagens e preservar os arquivos necessários à documentação Swagger.

O arquivo `docker-compose.portainer.yml` foi ajustado para utilizar as imagens publicadas, sem build local, selecionando a versão pela variável `IMAGE_TAG`. Foi adotada a modalidade “quase automática”, com testes, builds e publicação automáticos e atualização manual no Portainer. A stack existente foi atualizada

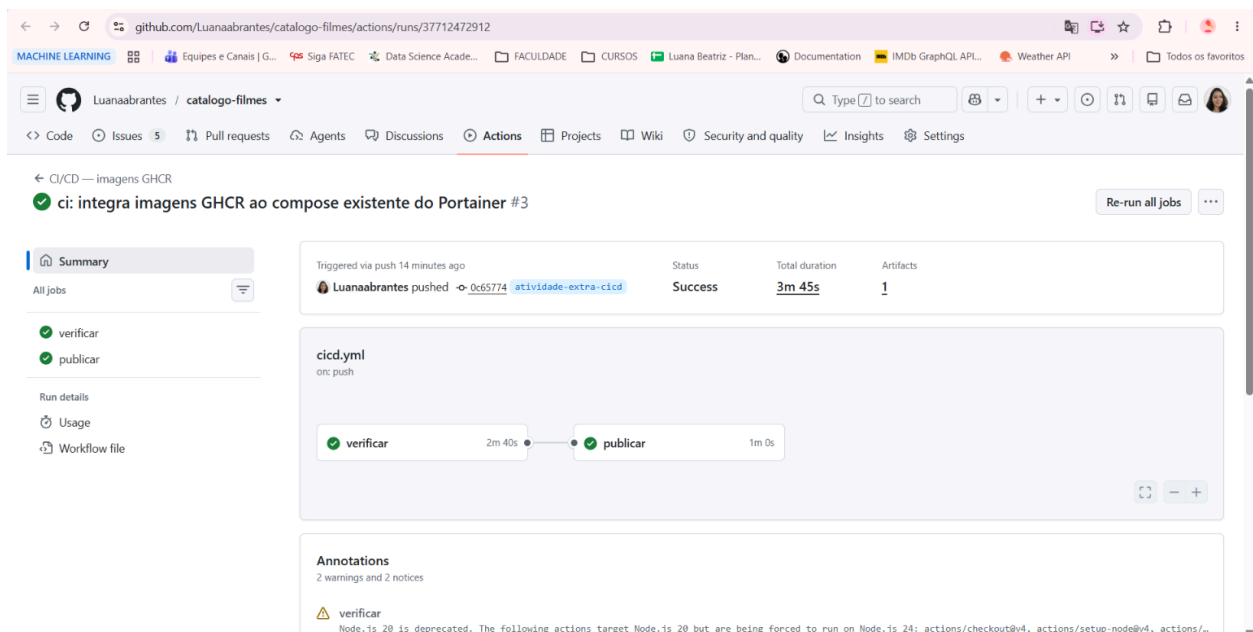
com a tag do commit 0c65774, preservando portas, rede, variáveis e volumes do MinIO e Redis. A execução do GitHub Actions terminou com os jobs verificar e publicar aprovados, incluindo 52 testes e a validação OpenAPI. Após o redeploy, os containers ficaram em execução ou saudáveis, e a página de perfil carregou com nome, foto e bio. O README foi atualizado com instruções de implantação e rollback; a automação completa do deploy permaneceu como melhoria futura.

Execução aprovada: [GitHub Actions — execução 37712472912](#).



```
luana@LuanaAbrantes MINGW64 ~/catalogo-filmes (atividade-extra-cicd)
$ git --no-pager log --reverse --date=format-local:"%d/%m/%Y %H:%M:%S" --pretty=format:"%h | %ad | %s" atividade-extra-swagger..0c65774
24abe30 | 07/10/2026 21:44:51 | ci: adiciona verificacao e publicacao de imagens no GHCR
3b89156 | 07/10/2026 21:50:07 | docs: registra execucao aprovada do pipeline CI/CD
0c65774 | 07/10/2026 22:20:50 | ci: integra imagens GHCR ao compose existente do Portainer
luana@LuanaAbrantes MINGW64 ~/catalogo-filmes (atividade-extra-cicd)
$
```

Figura 24: Histórico de commits da atividade extra de CI/CD, com implementação do pipeline, registro da execução aprovada e integração ao Portainer.



github.com/LuanaAbrantes/catalogo-filmes/actions/runs/37712472912

MACHINE LEARNING Equipes e Canais | G... Siga FATEC Data Science Acade... FACULDADE CURSOS Luana Beatriz - Plan... Documentation IMDb GraphQL API... Weather API Todos os favoritos

LuanaAbrantes / catalogo-filmes

<> Code Issues Pull requests Agents Discussions Actions Projects Wiki Security and quality Insights Settings

CI/CD — imagens GHCR

✓ ci: integra imagens GHCR ao compose existente do Portainer #3 Re-run all jobs

Summary

All jobs

verificar
publicar

Run details
Usage
Workflow file

Triggered via push 14 minutes ago

LuanaAbrantes pushed · 0c65774 atividade-extra-cicd

Status Success Total duration 3m 45s Artifacts 1

cicd.yml
on: push

verificar 2m 40s → publicar 1m 0s

Annotations
2 warnings and 2 notices

verificar
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@v4, actions/...

Figura 25: Execução do pipeline no GitHub Actions concluída com sucesso, com os jobs de verificação e publicação das imagens no GHCR aprovados.

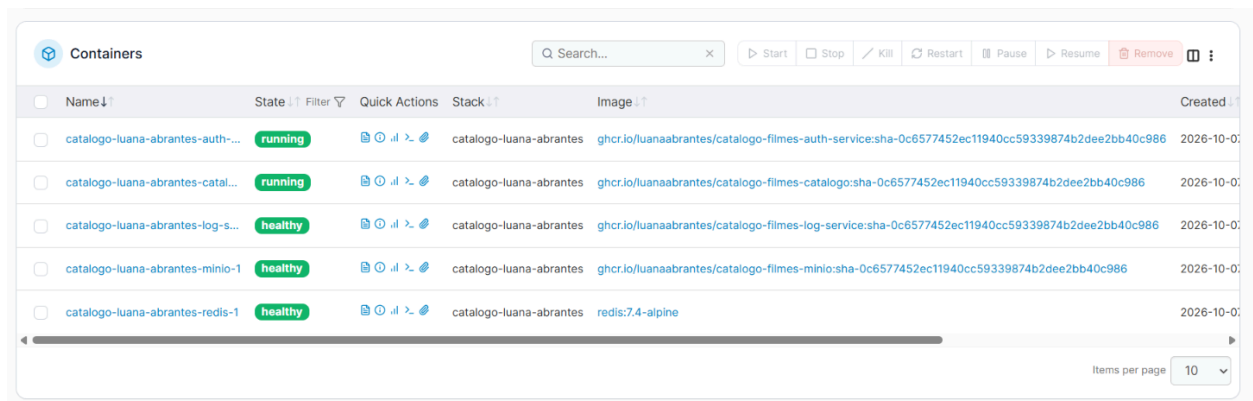


Figura 26: Containers da aplicação em execução no Portainer após a atualização com as imagens publicadas pelo pipeline CI/CD no GHCR.

Dificuldades e como foram resolvidas.

A principal dificuldade foi compreender como utilizar no Portainer as imagens publicadas pelo GitHub Actions. Foi necessário identificar a relação entre a branch do repositório, o arquivo Compose utilizado pela stack e a tag que determina a versão das imagens. A solução foi ajustar o `docker-compose.portainer.yml` para utilizar as quatro imagens do GHCR, mantendo o caminho já configurado na stack e preservando portas, rede, variáveis de ambiente e volumes persistentes.

Também foi necessário configurar a variável `IMAGE_TAG` no Portainer com a tag SHA completa do commit aprovado no pipeline. Após selecionar a branch `atividade-extra-cicd`, foi realizada a atualização por meio de Pull and redeploy. O resultado foi conferido pelos estados dos containers, pelas imagens e tags exibidas no Portainer e pelo carregamento da página de perfil com nome, foto e bio.

5.3 Atividade E3 — Observabilidade — health checks e métricas

Descrição	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.
Data planejada	sem prazo
Data realizada	07/10/2026 23:15:00 (UTC-03:00)
Situação	entregue
Evidência	Commit <code>d672301f5defa185e96ba3118f2605ca12057cd2</code> , execução aprovada do GitHub Actions e capturas da validação no laboratório isolado.
Link	https://github.com/Luanaabrantes/catalogo-filmes/commit/d672301f5defa185e96ba3118f2605ca12057cd2

O que foi feito.

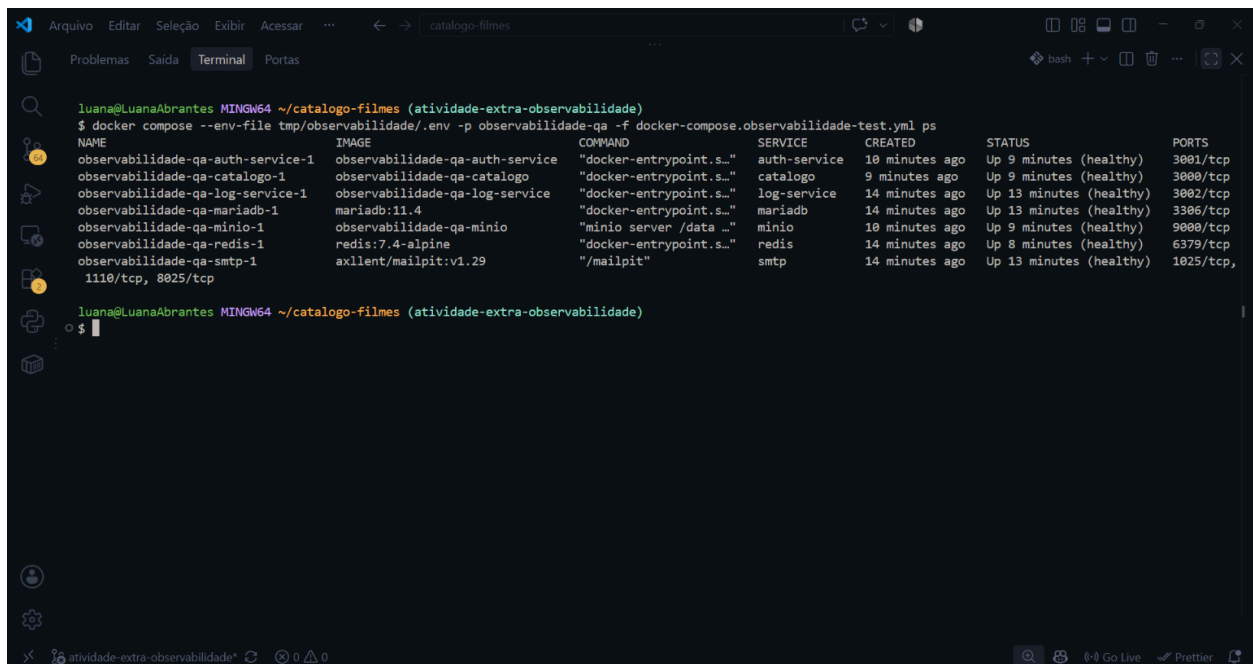
Foram implementados health checks reais no catálogo, no auth-service e no log-service. A rota `GET /live` verifica somente se o processo responde, enquanto `GET /health` consulta as dependências e retorna HTTP 200 quando o serviço está pronto ou HTTP 503 quando uma dependência falha. O catálogo verifica MariaDB, autenticação, MinIO e TMDb; o serviço de autenticação verifica MariaDB e SMTP; e o serviço de logs verifica Redis e autenticação. Os probes utilizam timeouts curtos e retornam informações de estado sem expor credenciais ou detalhes sensíveis.

Os Dockerfiles e os arquivos Compose foram ajustados para consultar `/health`, preservando portas, redes, volumes e o fluxo existente de CI/CD. No catálogo, a biblioteca `prom-client` disponibiliza `GET /metrics`, com contador de requisições e histograma de latência por método, template de rota e código de status. As

métricas registram também respostas de erro e evitam criar séries diferentes para IDs ou parâmetros de consulta. A documentação Swagger/OpenAPI foi atualizada para os novos endpoints.

A validação foi realizada em um laboratório Docker isolado, sem implantação da observabilidade em produção. Foram aprovados 56 testes automatizados e a validação OpenAPI. Os serviços ficaram healthy; ao interromper somente o Redis, o log-service passou a unhealthy, com /health retornando HTTP 503 e /live retornando HTTP 200. Após restaurar o Redis, o serviço de logs recuperou automaticamente o estado healthy, sem reinício manual. Também foram verificadas as métricas reais do catálogo. A execução do GitHub Actions concluiu as verificações com sucesso; a publicação foi ignorada nessa branch, conforme a restrição existente à branch de CI/CD.

Execução aprovada: [GitHub Actions](#) — execução 37716954218.



The screenshot shows a terminal window with the command `docker compose --env-file tmp/observabilidade/.env -p observabilidade-qa -f docker-compose.observabilidade-test.yml ps` executed. The output is a table listing the status of various Docker services.

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
observabilidade-qa-auth-service-1	observabilidade-qa-auth-service	"docker-entrypoint.s..."	auth-service	10 minutes ago	Up 9 minutes (healthy)	3801/tcp
observabilidade-qa-catalogo-1	observabilidade-qa-catalogo	"docker-entrypoint.s..."	catalogo	9 minutes ago	Up 9 minutes (healthy)	3800/tcp
observabilidade-qa-log-service-1	observabilidade-qa-log-service	"docker-entrypoint.s..."	log-service	14 minutes ago	Up 13 minutes (healthy)	3802/tcp
observabilidade-qa-mariadb-1	mariadb:11.4	"docker-entrypoint.s..."	mariadb	14 minutes ago	Up 13 minutes (healthy)	3306/tcp
observabilidade-qa-minio-1	observabilidade-qa-minio	"minio server /data -"	minio	10 minutes ago	Up 9 minutes (healthy)	9000/tcp
observabilidade-qa-redis-1	redis:7.4-alpine	"docker-entrypoint.s..."	redis	14 minutes ago	Up 8 minutes (healthy)	6379/tcp
observabilidade-qa-smtp-1	axllent/mailpit:v1.29	"/mailpit"	smtp	14 minutes ago	Up 13 minutes (healthy)	1025/tcp, 8025/tcp

Figura 27: Serviços do laboratório com health checks em estado healthy.

```

luana@luanaabrant: ~/catalogo-filmes
$ docker compose --env-file tmp/observabilidade/.env -p observabilidade-qa -f docker-compose.observabilidade-test.yml ps -a
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS                PORTS
observabilidade-qa-auth-service-1    observabilidade-qa-auth-service      "docker-entrypoint.s..."          auth-service  14 minutes ago Up 13 minutes (healthy) 3001/tcp
observabilidade-qa-catalogo-1        observabilidade-qa-catalogo          "docker-entrypoint.s..."          catalogo      14 minutes ago Up 13 minutes (healthy) 3000/tcp
observabilidade-qa-log-service-1      observabilidade-qa-log-service        "docker-entrypoint.s..."          log-service   18 minutes ago Up 17 minutes (unhealthy) 3002/tcp
observabilidade-qa-mariadb-1          mariadb:11.4                         "docker-entrypoint.s..."          mariadb       18 minutes ago Up 18 minutes (healthy) 3306/tcp
observabilidade-qa-minio-1            observabilidade-qa-minio             "minio server /data ..."          minio         14 minutes ago Up 13 minutes (healthy) 9000/tcp
observabilidade-qa-redis-1            redis:7.4-alpine                     "docker-entrypoint.s..."          redis         18 minutes ago Exited (0) About a minute ago
observabilidade-qa-smtp-1             axllent/mailpit:v1.29                "/mailpit"                          smtp          18 minutes ago Up 18 minutes (healthy) 1025/tcp, 1110/tcp, 8025/tcp

luana@luanaabrant: ~/catalogo-filmes
$ docker compose --env-file tmp/observabilidade/.env -p observabilidade-qa -f docker-compose.observabilidade-test.yml exec -T log-service node -e "fetch('http://127.0.0.1:3002/health').then(async r => console.log(r.status, await r.json()))"
503 {
  service: 'log-service',
  status: 'indisponivel',
  dependencias: { redis: 'indisponivel', autenticacao: 'ok' }
}

luana@luanaabrant: ~/catalogo-filmes
$ docker compose --env-file tmp/observabilidade/.env -p observabilidade-qa -f docker-compose.observabilidade-test.yml exec -T log-service node -e "fetch('http://127.0.0.1:3002/live').then(async r => console.log(r.status, await r.json()))"
200 { service: 'log-service', status: 'ok' }

luana@luanaabrant: ~/catalogo-filmes
$

```

Figura 28: Redis interrompido: log-service unhealthy, /health com HTTP 503 e /live com HTTP 200.

```

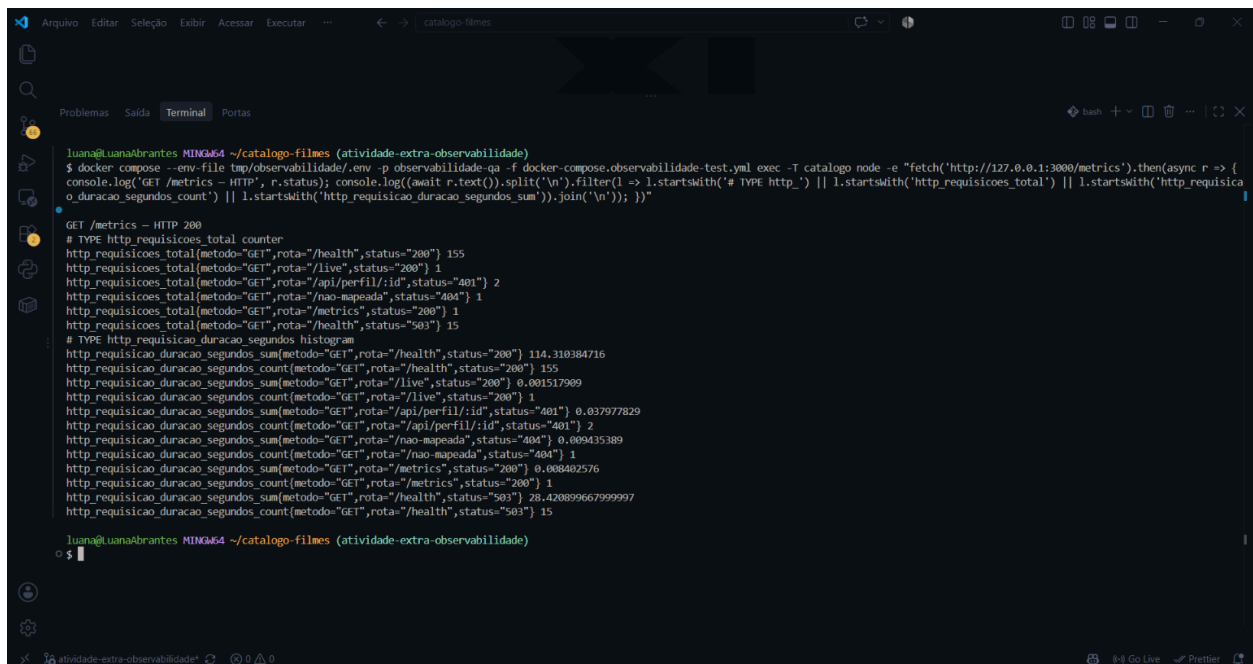
luana@luanaabrant: ~/catalogo-filmes
$ docker compose --env-file tmp/observabilidade/.env -p observabilidade-qa -f docker-compose.observabilidade-test.yml start redis
[+] start 1/1
✓ container observabilidade-qa-redis-1 Started 1.7s

luana@luanaabrant: ~/catalogo-filmes
$ docker compose --env-file tmp/observabilidade/.env -p observabilidade-qa -f docker-compose.observabilidade-test.yml ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS                PORTS
observabilidade-qa-auth-service-1    observabilidade-qa-auth-service      "docker-entrypoint.s..."          auth-service  17 minutes ago Up 17 minutes (healthy) 3001/tcp
observabilidade-qa-catalogo-1        observabilidade-qa-catalogo          "docker-entrypoint.s..."          catalogo      17 minutes ago Up 17 minutes (healthy) 3000/tcp
observabilidade-qa-log-service-1      observabilidade-qa-log-service        "docker-entrypoint.s..."          log-service   21 minutes ago Up 21 minutes (healthy) 3002/tcp
observabilidade-qa-mariadb-1          mariadb:11.4                         "docker-entrypoint.s..."          mariadb       21 minutes ago Up 21 minutes (healthy) 3306/tcp
observabilidade-qa-minio-1            observabilidade-qa-minio             "minio server /data ..."          minio         17 minutes ago Up 17 minutes (healthy) 9000/tcp
observabilidade-qa-redis-1            redis:7.4-alpine                     "docker-entrypoint.s..."          redis         21 minutes ago Up 46 seconds (healthy) 6379/tcp
observabilidade-qa-smtp-1             axllent/mailpit:v1.29                "/mailpit"                          smtp          21 minutes ago Up 21 minutes (healthy) 1025/tcp, 1110/tcp, 8025/tcp

luana@luanaabrant: ~/catalogo-filmes
$

```

Figura 29: Recuperação do log-service para healthy após restaurar o Redis, sem reiniciar o serviço de logs.



```
Luana@luanaabranes MINGW64 ~/catalogo-filmes (atividade-extra-observabilidade)
$ docker compose --env-file tmp/observabilidade/.env -p observabilidade-qa -f docker-compose.observabilidade-test.yml exec -T catalogo node -e "fetch('http://127.0.0.1:3000/metrics').then(async r => { console.log('GET /metrics - HTTP', r.status); console.log(await r.text().split('\n').filter(l => l.startsWith('# TYPE http_') || l.startsWith('http_requisicoes_total') || l.startsWith('http_requisicao_duracao_segundos_count') || l.startsWith('http_requisicao_duracao_segundos_sum')).join('\n')); })"

GET /metrics - HTTP 200
# TYPE http_requisicoes_total counter
http_requisicoes_total{metodo="GET",rota="/health",status="200"} 155
http_requisicoes_total{metodo="GET",rota="/live",status="200"} 1
http_requisicoes_total{metodo="GET",rota="/api/perfil/id",status="401"} 2
http_requisicoes_total{metodo="GET",rota="/nao-mapeada",status="404"} 1
http_requisicoes_total{metodo="GET",rota="/metrics",status="200"} 1
http_requisicoes_total{metodo="GET",rota="/health",status="503"} 15
# TYPE http_requisicao_duracao_segundos histogram
http_requisicao_duracao_segundos_sum{metodo="GET",rota="/health",status="200"} 114.310384716
http_requisicao_duracao_segundos_count{metodo="GET",rota="/health",status="200"} 155
http_requisicao_duracao_segundos_sum{metodo="GET",rota="/live",status="200"} 0.001517909
http_requisicao_duracao_segundos_count{metodo="GET",rota="/live",status="200"} 1
http_requisicao_duracao_segundos_sum{metodo="GET",rota="/api/perfil/id",status="401"} 0.037977829
http_requisicao_duracao_segundos_count{metodo="GET",rota="/api/perfil/id",status="401"} 2
http_requisicao_duracao_segundos_sum{metodo="GET",rota="/nao-mapeada",status="404"} 0.009435389
http_requisicao_duracao_segundos_count{metodo="GET",rota="/nao-mapeada",status="404"} 1
http_requisicao_duracao_segundos_sum{metodo="GET",rota="/metrics",status="200"} 0.008402576
http_requisicao_duracao_segundos_count{metodo="GET",rota="/metrics",status="200"} 1
http_requisicao_duracao_segundos_sum{metodo="GET",rota="/health",status="503"} 28.420899667999997
http_requisicao_duracao_segundos_count{metodo="GET",rota="/health",status="503"} 15

Luana@luanaabranes MINGW64 ~/catalogo-filmes (atividade-extra-observabilidade)
$
```

Figura 30: Endpoint /metrics com HTTP 200, contador de requisições e histograma de latência por método, rota e status.

Dificuldades e como foram resolvidas.

Foi necessário separar a disponibilidade do processo da prontidão para atender requisições e evitar dependências circulares entre os serviços. A solução foi manter /live independente e consultar apenas as dependências necessárias em /health, com timeouts e respostas de estado. A auditoria permaneceu de melhor esforço, permitindo que o catálogo continuasse pronto durante a interrupção do Redis. O teste utilizou uma stack separada com banco, Redis e MinIO próprios, preservando o ambiente de produção.

Outro ajuste foi garantir que as métricas utilizassem o template da rota mesmo quando o middleware de autenticação encerrasse a requisição antes do handler. Os contratos OpenAPI foram usados para identificar essas rotas, permitindo agrupar as respostas negadas sem incluir IDs nos labels. Testes automatizados e requisições reais no laboratório confirmaram o contador, o histograma e a recuperação do serviço de logs após a restauração do Redis.

6 Considerações finais

O principal aprendizado deste bimestre foi compreender o fluxo entre o desenvolvimento da aplicação e sua disponibilização em um servidor. Antes das atividades, eu não conhecia algumas das ferramentas utilizadas e imaginava que colocar um sistema no ar seria mais difícil. Ao trabalhar com Docker e Portainer, pude entender como uma aplicação desenvolvida localmente pode ser publicada e ficar disponível para acesso pela internet.

Encarar as atividades como desafios também tornou o aprendizado mais interessante. A cada semana, foi possível acompanhar o catálogo de filmes tomando forma com a implementação de novas funcionalidades. A autenticação, o controle de acesso, a auditoria e o perfil com upload de imagem ajudaram a compreender como diferentes serviços se relacionam e contribuem para o funcionamento de uma mesma aplicação.

As atividades extras ampliaram esse entendimento ao abordar a documentação das APIs, a automação de testes e publicação de imagens e a verificação da saúde dos serviços. Com isso, passei a perceber a importância de verificar o funcionamento da aplicação, registrar suas interfaces e acompanhar possíveis falhas, além de implementar as funcionalidades solicitadas.

As aulas foram de grande importância para ampliar meus conhecimentos e relacionar os conceitos apresentados com a prática. As explicações e orientações do professor ajudaram a esclarecer dúvidas e a avançar nas etapas do projeto, principalmente no contato com ferramentas e procedimentos que eram novos para mim.

Ao final do bimestre, considero que o desenvolvimento gradual do projeto contribuiu para aumentar minha confiança e autonomia. Ainda há conhecimentos a aprofundar, mas a experiência permitiu compreender melhor as etapas necessárias para desenvolver, publicar e acompanhar uma aplicação em nuvem, formando uma base para as próximas atividades e outros projetos.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Luana Beatriz Ribeiro Abrantes

Pompeia, 07/10/2026